

**LAPORAN SISTEM MONITORING IOT
PENGENDALIAN OBJEK VIRTUAL MENGGUNAKAN
SENSOR IR OBSTACLE BERBASIS WEB**



Nama Anggota Kelompok 7 :

Muhammad Rifqi Alimin 20230140147

Eko Ramadhan Nugroho 20230140114

Abiem Muhammad Resky 20230140124

UNIVERSITAS MUHAMMADIYAH YOGYAKARTA

FAKULTAS TEKNIK

PROGRAM STUDI TEKNOLOGI INFORMASI

2025/2026

DAFTAR ISI

BAB 1 PENDAHULUAN	2
1.1 Latar Belakang.....	2
1.2 Rumusan Masalah	2
1.3 Tujuan Penelitian	2
1.4 Manfaat	3
BAB 2 UML.....	3
2.1 Use Case	4
2.2 Flow Chart	4
2.2.1 User.....	5
2.2.2 Admin	5
2.3 ERD	6
2.4 Class Diagram.....	7
2.5 Arsitektur Diagram	8
2.6 Activity Diagram	10
2.7 Sequence Diagram	11
BAB 3 IMPLEMENTASI APLIKASI	12
3.1 Foto Alat (Perangkat Keras).....	13
3.2 I Tampilan Web.....	13
3.3 Penjelasan Susunan Folder dan Source Code	13
3.4 Implementasi Antarmuka (Web Admin)	14
BAB 4 PENUTUP.....	14
4.1 Kesimpulan.....	14
4.2 Saran.....	14
DAFTAR PUSTAKA	14
LAMPIRAN	15
Frontend Logic (JavaScript, Html, Css).....	16
Firmware ESP32 (C++)	30

BAB 1

PENDAHULUAN

1.1 Latar Belakang

Perkembangan teknologi Internet of Things (IoT) memungkinkan pengendalian dan pemantauan perangkat jarak jauh secara real-time. Salah satu penerapannya adalah pada sistem keamanan atau kendali otomatis yang minim sentuhan (touchless). Penggunaan saklar fisik konvensional memiliki risiko kerusakan mekanis dan kurang higienis. Oleh karena itu, diperlukan sistem kendali berbasis sensor jarak (obstacle sensor) yang terintegrasi dengan internet.

Masalah yang sering terjadi pada sistem IoT adalah ketidakpastian status perangkat saat terjadi gangguan daya atau koneksi. Pengguna seringkali tidak mengetahui apakah perangkat sedang "mati" (offline) atau sekadar tidak mendeteksi objek. Proyek ini bertujuan membangun sistem monitoring objek virtual yang dilengkapi dengan sinkronisasi waktu (NTP) dan fitur deteksi status offline (Heartbeat) menggunakan Firebase Realtime Database.

1.2 Rumusan Masalah

Berdasarkan latar belakang di atas, rumusan masalah dalam penelitian ini adalah:

1. Bagaimana merancang sistem kendali objek virtual menggunakan ESP32 dan Sensor IR Obstacle?
2. Bagaimana mengimplementasikan sinkronisasi waktu nyata (NTP) agar log data akurat?
3. Bagaimana mendeteksi status online/offline perangkat pada dashboard web menggunakan logika Heartbeat?

1.3 Tujuan Penelitian

1. Membuat purwarupa alat IoT pendeteksi halangan berbasis ESP32.
2. Membangun dashboard web yang dapat menampilkan status visual dan waktu update secara *real-time*.
3. Menerapkan fitur keamanan login admin dan notifikasi status perangkat mati.

1.4 Manfaat

Sistem ini dapat diterapkan sebagai dasar pengembangan *Smart Home*, sistem keamanan pintu otomatis, atau penghitung jumlah barang/orang dengan pemantauan terpusat yang handal.

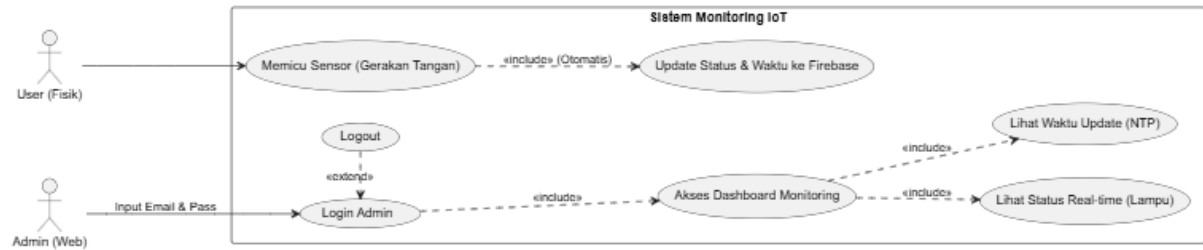
BAB 2

UML

2.1 Use Case

Terdapat dua aktor utama:

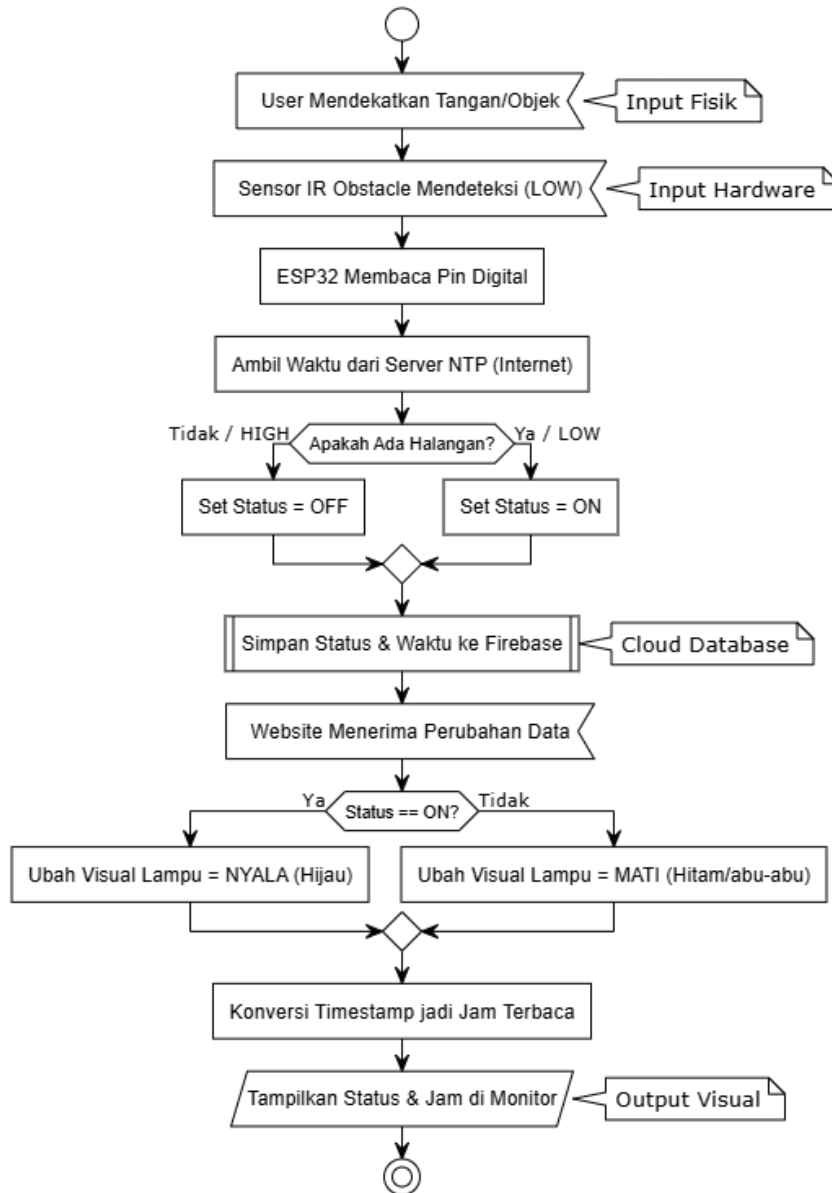
Aktor dalam sistem ini dibagi menjadi User Fisik (Interaksi Hardware) dan Admin (Interaksi Web).



2.2 Flow Chart

2.2.1 User

ESP32 membaca sinyal IR -> Validasi Kode -> Kirim HTTP POST ke Firebase.



2.2.2 Admin

Login -> Masuk Dashboard -> Mulai Timer (Interval 3 Detik) -> Request Data ke Firebase -> Update Tampilan.

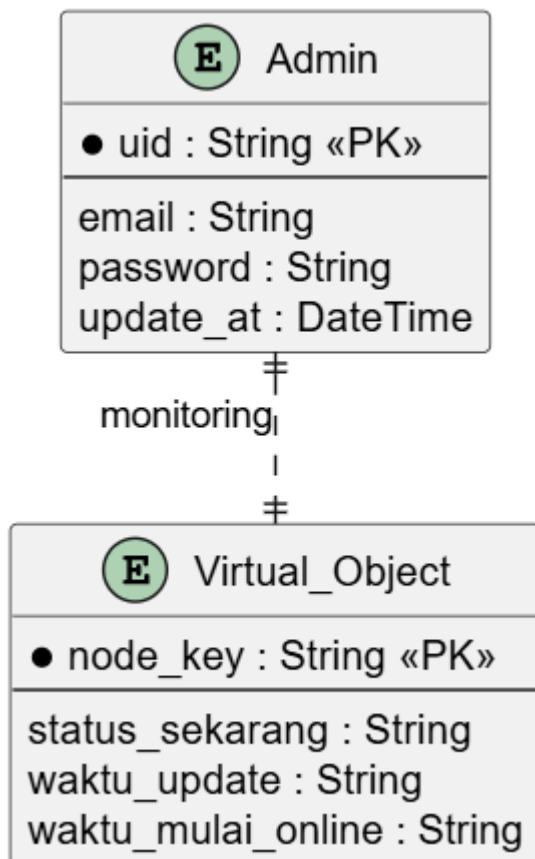
Flowchart Admin (Monitoring Only)



2.3 ERD

Diagram di atas memodelkan struktur data untuk fitur pemantauan sistem. Terdapat dua entitas utama:

1. **Admin:** Berfungsi menyimpan data otentikasi pengguna, meliputi `uid` sebagai kunci utama, `email`, dan `password`.
2. **Virtual_Object:** Berfungsi menyimpan status perangkat atau *node* yang dipantau, meliputi identitas unik (`node_key`), status terkini, dan log waktu aktivitas.

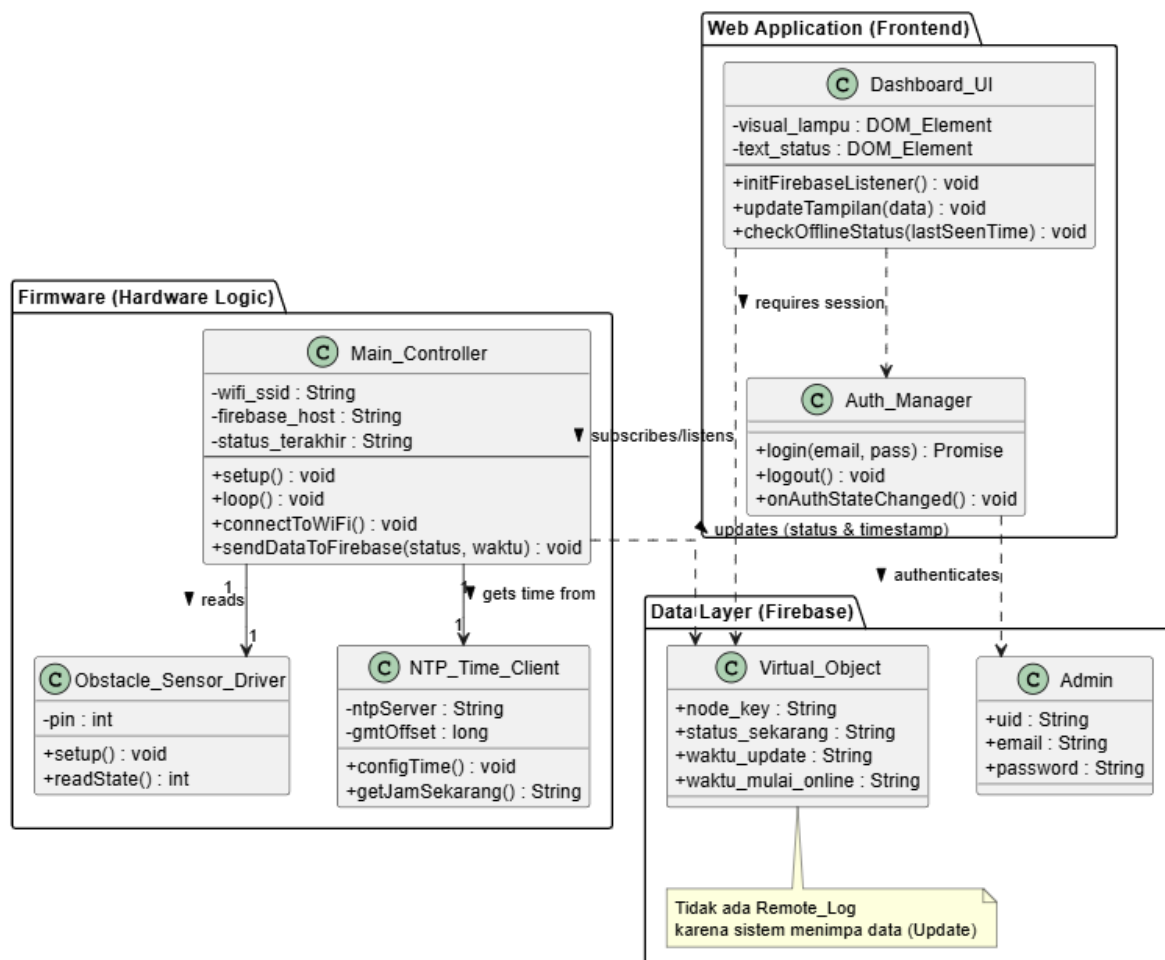


2.4 Class Diagram

Diagram kelas menggambarkan struktur kode program dan model data yang dibangun. Sistem ini dibagi menjadi tiga paket utama:

1. Firmware (ESP32): Menggunakan pendekatan modular dengan memisahkan fungsi `Obstacle_Sensor_Driver` untuk pembacaan sinyal digital sensor dan `NTP_Time_Client` untuk melakukan sinkronisasi waktu dengan server internet.
2. Web Application (Frontend): Menerapkan pola desain sederhana dimana `Auth_Manager` menangani sesi login/logout pengguna, dan `Dashboard_UI` bertanggung jawab memanipulasi elemen DOM (Document Object Model) serta menjalankan algoritma deteksi offline.
3. Data Layer (Firebase Model): Merepresentasikan skema data di cloud. Terdiri dari class `Admin` untuk menyimpan data autentikasi pengguna, dan class `Virtual_Object` yang menyimpan status sensor serta log waktu (timestamp) secara real-time tanpa menyimpan riwayat historis (sistem override).

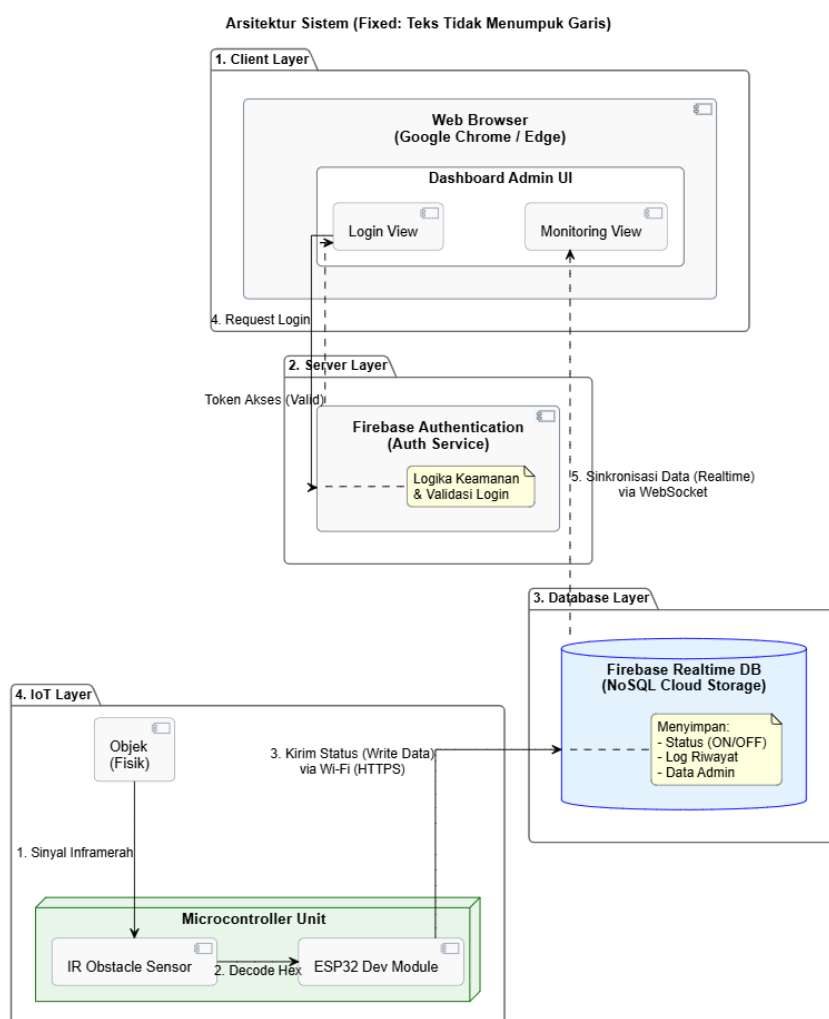
Class Diagram: Sistem Monitoring IoT (ESP32 + Obstacle + NTP)



2.5 Arsitektur Diagram

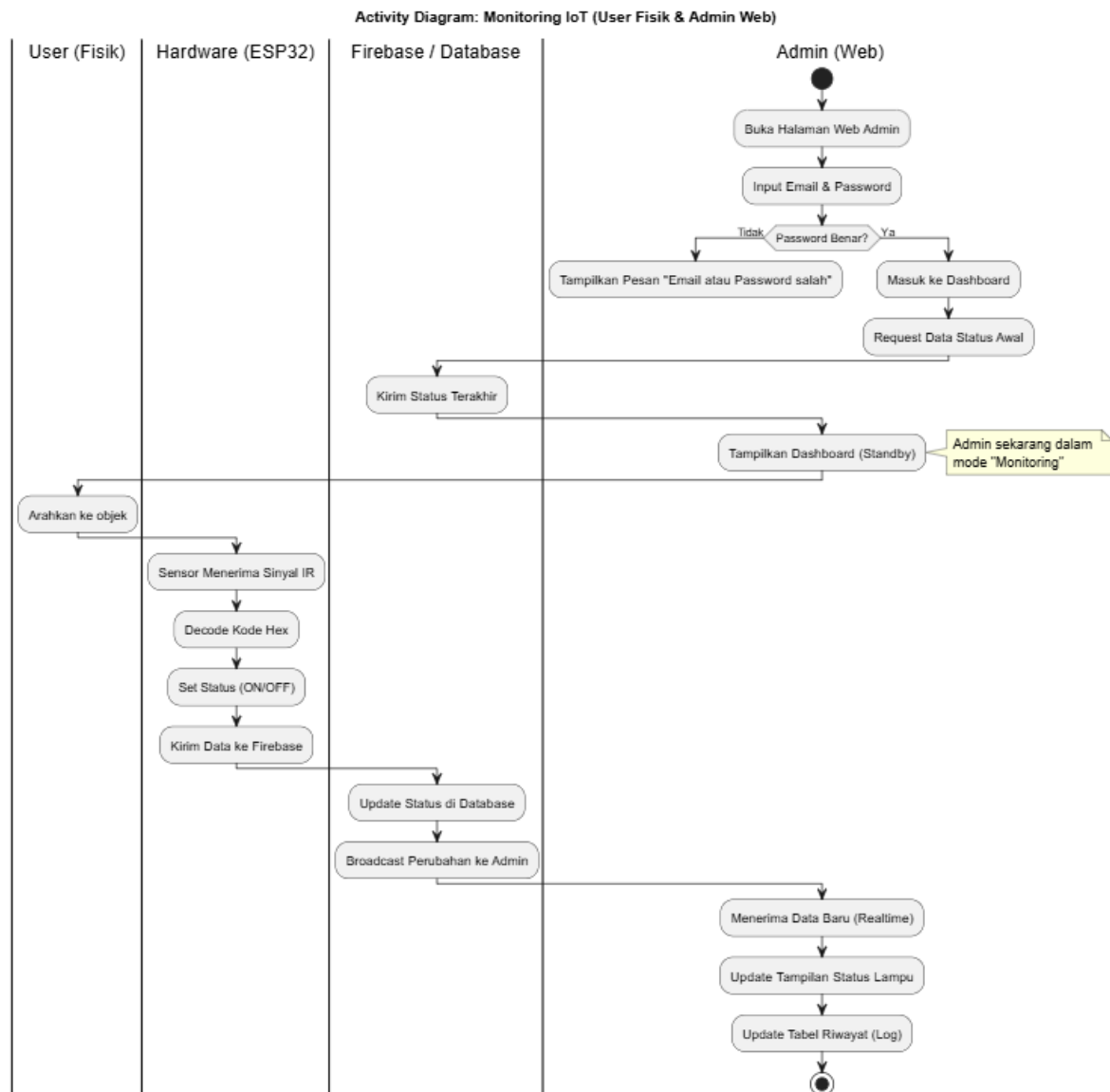
Arsitektur sistem menggambarkan rancangan topologi jaringan dan komponen yang saling terhubung dalam membangun sistem monitoring objek virtual ini. Sistem ini terdiri dari tiga lapisan utama (three-tier architecture), yaitu:

1. **Client Layer (Sisi Pengguna):** Merupakan antarmuka visual berupa Dashboard Web yang diakses oleh Admin menggunakan web browser. Lapisan ini berfungsi menampilkan status sensor (ON/OFF) dan waktu update secara real-time, serta menjalankan logika deteksi status offline.
2. **Server & Database Layer (Firebase):** Bertindak sebagai jembatan penghubung (middleware). Firebase Authentication menangani keamanan login, sedangkan Firebase Realtime Database berfungsi sebagai media penyimpanan sementara yang menyinkronkan data antara perangkat keras dan website dengan latensi rendah.
3. **IoT Device Layer (Sisi Perangkat):** Terdiri dari mikrokontroler ESP32 dan sensor IR Obstacle. Lapisan ini bertugas membaca kondisi fisik di lapangan, mengambil waktu dari server NTP (Network Time Protocol), dan mengirimkan data ke cloud



2.6 Activity Diagram

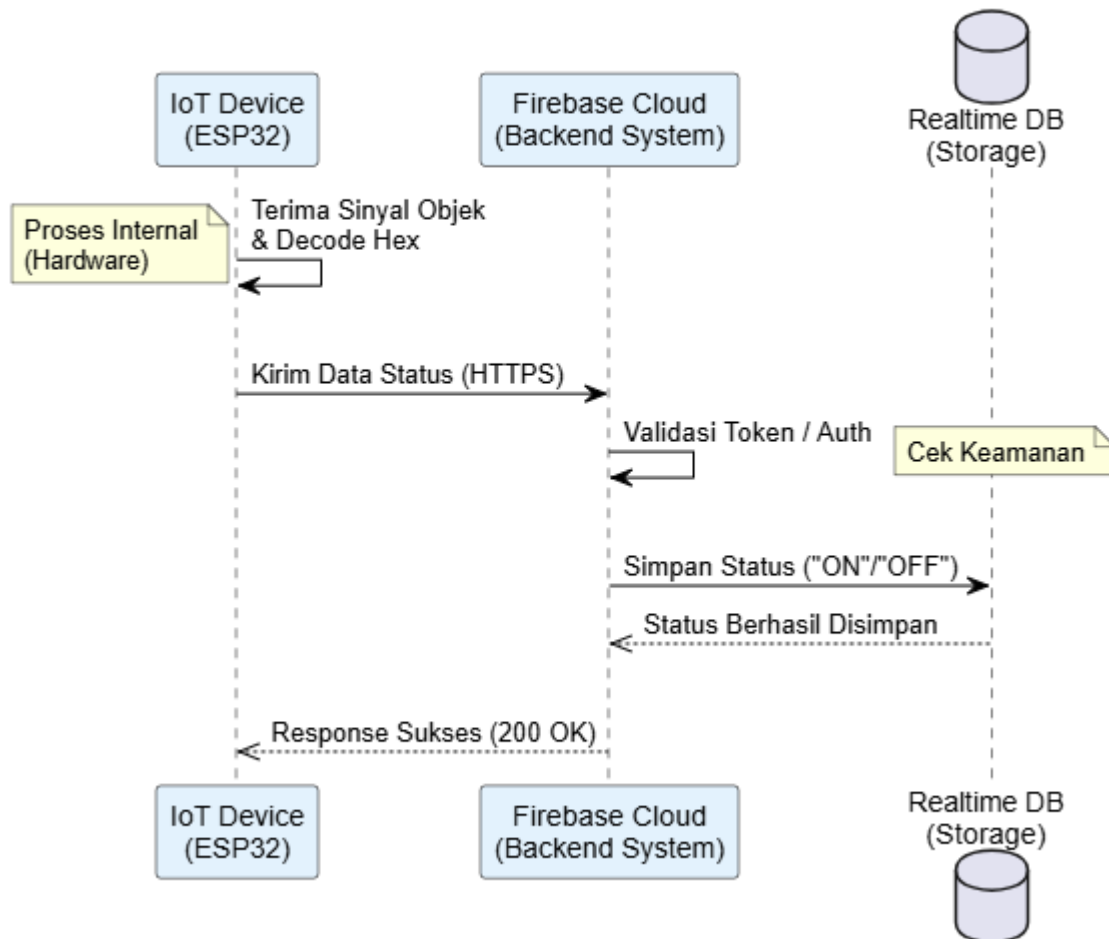
Alur kerja sistem dimulai dari sensor tangan. Sinyal diproses oleh ESP32 dan dikirim ke Firebase. Di sisi lain, Web Admin secara aktif melakukan pengecekan (request) ke database setiap interval waktu tertentu untuk mengambil status terbaru dan memperbarui tampilan.



2.7 Sequence Diagram

Sequence diagram menggambarkan urutan pertukaran pesan antar objek dalam dimensi waktu. Diagram ini menunjukkan bagaimana data status dikirim dari ESP32 ke Cloud, divalidasi keamanannya, lalu disimpan ke Realtime Database. Respon sukses (200 OK) dikirimkan kembali ke ESP32 sebagai konfirmasi bahwa data telah tersimpan.

Sequence Diagram: Pengiriman Data (Tanpa Activation Bar)



BAB 3

IMPLEMENTASI APLIKASI

3.1 Foto Alat (Perangkat Keras)

Sistem menggunakan mikrokontroler ESP32 DevKit V1 yang terhubung dengan Sensor IR Obstacle pada pin GPIO 15.

3.2 I Tampilan Web

Berikut adalah implementasi antarmuka dashboard admin.

A. Halaman Login Halaman ini membatasi akses hanya untuk admin yang terdaftar menggunakan Firebase Authentication. (Tempelkan Screenshot Halaman Login)

B. Dashboard Utama (Status Online) Menampilkan indikator visual lampu (Kuning=ON, Abu-abu=OFF) dan waktu update terakhir yang disinkronisasi dengan server NTP. (Tempelkan Screenshot Dashboard saat alat menyala)

C. Dashboard (Status Offline/Mati) Sistem menampilkan peringatan merah jika ESP32 tidak mengirim sinyal heartbeat selama lebih dari 15 detik. (Tempelkan Screenshot Dashboard saat alat mati/offline)

3.3 Penjelasan Susunan Folder dan Source Code

Struktur direktori proyek web adalah sebagai berikut:

Plaintext

PROJECT_IOT/

├── index.html	(Struktur Halaman Web: Login & Dashboard)
├── style.css	(Desain Tampilan: Tata letak, warna, animasi lampu)
└── script.js	(Logika Program: Koneksi Firebase, Auth, Deteksi Offline)

1. index.html: Berfungsi sebagai kerangka utama. Menggunakan div tersembunyi (hidden) untuk memisahkan tampilan Login dan Dashboard dalam satu file (Single Page Application).
2. script.js: Menggunakan module ES6 untuk mengimpor Firebase SDK. Berisi fungsi updateTampilan() yang memiliki algoritma pembandingan waktu untuk mendeteksi apakah alat masih online atau sudah mati.
3. style.css: Mengatur responsivitas dan efek visual nyala/mati pada indikator lampu.

3.4 Implementasi Antarmuka (Web Admin)

Halaman web dibangun menggunakan HTML5 dan JavaScript murni (Native).

1. Halaman Login: Admin memasukkan email dan password. Sistem memverifikasi ke Firebase Auth.
2. Dashboard Monitoring: Setelah login, halaman ini menampilkan visualisasi lampu yang berubah warna sesuai status ON atau OFF yang diterima dari database secara real-time tanpa refresh.

BAB 4 PENUTUP

4.1 Kesimpulan

Berdasarkan hasil perancangan dan implementasi yang telah dilakukan, dapat disimpulkan bahwa:

1. Sistem berhasil menghubungkan interaksi fisik (remote control) dengan dunia digital (web) menggunakan perantara ESP32 dan Firebase.
2. Arsitektur sistem yang terbagi menjadi 4 layer memudahkan pengembangan dan pemeliharaan sistem.
3. Penggunaan teknologi WebSocket pada Firebase terbukti efektif memberikan pengalaman pemantauan yang responsif (real-time).

4.2 Saran

Untuk pengembangan selanjutnya, disarankan agar:

1. Menambahkan *Buzzer* pada perangkat keras sebagai indikator suara saat status berubah.
2. Mengembangkan aplikasi mobile (Android) agar notifikasi bisa muncul di *status bar* smartphone.
3. Menambahkan baterai cadangan pada alat agar tetap beroperasi saat listrik utama padam.

DAFTAR PUSTAKA

1. Espressif Systems. (2024). *ESP32 Technical Reference Manual*.
2. Google Developers. (2024). *Firebase Documentation: Realtime Database & Authentication*.
3. Pressman, R. S. (2019). *Software Engineering: A Practitioner's Approach*. McGraw-Hill Education.

LAMPIRAN

Frontend Logic (JavaScript, Html, Css)

Script.js :

```
// 1. IMPORT LIBRARY FIREBASE

import { initializeApp } from
"https://www.gstatic.com/firebasejs/10.7.1/firebase-
app.js";

import {
  getAuth,
  signInWithEmailAndPassword,
  onAuthStateChanged,
  signOut,
} from
"https://www.gstatic.com/firebasejs/10.7.1/firebase-
auth.js";

// 2. KONFIGURASI FIREBASE
// (PASTIKAN Bagian ini sudah sesuai dengan punya kamu
sendiri)
const firebaseConfig = {
  apiKey: "AIzaSyCFZeocCU94dHcg0DjE3JC4uumv0LDQ2E8",
  authDomain: "iot-sensor-saya.firebaseio.com",
  databaseURL:
```

```
    "https://iot-sensor-saya-default-rtdb.asia-southeast1.firebaseio.app",
    projectId: "iot-sensor-saya",
    storageBucket: "iot-sensor-saya.firebaseio.app",
    messagingSenderId: "930380131488",
    appId: "1:930380131488:web:c984fd6691b055b0226dc5",
  };
```

```
// 3. URL DATABASE (PERUBAHAN DISINI)
```

```
// Kita ambil folder "Virtual_Object" agar dapat Status DAN Waktu sekaligus.
```

```
const URL_DB_POLLING = firebaseConfig.databaseURL +
"/Virtual_Object.json";
```

```
// Inisialisasi Firebase
```

```
const app = initializeApp(firebaseConfig);
```

```
const auth = getAuth(app);
```

```
// Variabel Global
```

```
let pollingService;
```

```
// ===== FUNGSI LOGIN =====
```

```
window.prosesAuth = function () {  
    const email =  
document.getElementById("email").value;  
    const pass =  
document.getElementById("password").value;  
    const errorMsg = document.getElementById("error-  
msg");  
    const btn = document.getElementById("btn-submit");  
  
    errorMsg.innerText = "";  
    btn.innerText = "Loading...";  
    btn.disabled = true;  
  
    signInWithEmailAndPassword(auth, email, pass)  
        .then((userCredential) => {  
            console.log("Login Sukses:",  
userCredential.user.email);  
        })  
        .catch((error) => {  
            btn.innerText = "MASUK";  
            btn.disabled = false;  
            errorMsg.innerText =  
terjemahkanError(error.code);  
        });  
};
```

```
};

window.keluarAkun = function () {
  signOut(auth)
    .then(() => {
      console.log("User Logout");
    })
    .catch((error) => {
      console.error(error);
    });
};

function terjemahkanError(errorCode) {
  switch (errorCode) {
    case "auth/invalid-email":
      return "Format email salah.";
    case "auth/user-not-found":
      return "Akun tidak ditemukan.";
    case "auth/wrong-password":
    case "auth/invalid-credential":
      return "Email atau Password salah.";
    case "auth/missing-password":
      return "Password tidak boleh kosong.";
    default:
  }
}
```

```
        return "Error: " + errorCode;
    }
}

// ===== MONITOR STATUS USER
=====

onAuthStateChanged(auth, (user) => {
    const authSection = document.getElementById("auth-
section");
    const dashSection =
document.getElementById("dashboard-section");
    const btn = document.getElementById("btn-submit");

    if (user) {
        // USER LOGIN
        authSection.classList.add("hidden");
        dashSection.classList.remove("hidden");
        document.getElementById("user-email").innerText =
user.email;

        btn.disabled = false;
        btn.innerText = "MASUK";

        mulaiPollingData();
    }
}
```

```
    } else {  
        // USER LOGOUT  
        dashSection.classList.add("hidden");  
        authSection.classList.remove("hidden");  
        clearInterval(pollingService);  
    }  
});  
  
// ===== MONITOR DATA (POLLING)  
=====  
  
function mulaiPollingData() {  
    ambilData();  
    pollingService = setInterval(ambilData, 1000);  
}  
  
function ambilData() {  
    fetch(URL_DB_POLLING)  
        .then((response) => response.json())  
        .then((data) => {  
            updateTampilan(data);  
        })  
        .catch((error) => {  
            console.error("Gagal ambil data:", error);  
        });  
}
```

```
        document.getElementById("status-teks").innerText
= "OFFLINE";

    });
}

// FUNGSI UPDATE TAMPILAN (DIPERBAHARUI)
function updateTampilan(data) {
    const elemenLampu = document.getElementById("visual-
lampu");

    const elemenTeks = document.getElementById("status-
teks");

    const elemenWaktu = document.getElementById("last-
update");

    // Cek apakah data ada?
    if (data) {
        // 1. Ambil Status (Default OFF jika kosong)
        const status = data.status_sekarang || "OFF";

        // 2. Ambil Waktu & Konversi
        const timestamp = data.waktu_update;
        if (timestamp) {
            // Ubah angka detik (Unix) jadi Tanggal Manusia
```

```
        // Kalikan 1000 karena Javascript butuh  
milidetik  
  
        const dateObj = new Date(timestamp * 1000);  
  
        // Format Indonesia  
  
        elemenWaktu.innerText =  
dateObj.toLocaleString("id-ID");  
    } else {  
        elemenWaktu.innerText = "-";  
    }  
  
    // 3. Update Visual Lampu  
    if (status === "ON") {  
        elemenLampu.className = "lampu nyala";  
        elemenTeks.innerText = "STATUS: TERDETEKSI  
(ON)";  
        elemenTeks.style.color = "#d4a017";  
    } else {  
        elemenLampu.className = "lampu mati";  
        elemenTeks.innerText = "STATUS: KOSONG (OFF)";  
        elemenTeks.style.color = "#333";  
    }  
}  
}
```


Index.html :

```
<!DOCTYPE html>
<html lang="id">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width,
initial-scale=1.0" />
    <title>Sistem Monitoring IoT</title>
    <link rel="stylesheet" href="style.css" />
  </head>
  <body>
    <div id="auth-section" class="container">
      <h2 id="auth-title">Login Admin</h2>
      <p id="auth-desc">Silakan masuk untuk memantau
sistem.</p>

      <input
        type="email"
        id="email"
        placeholder="Email (Contoh: admin@gmail.com)"
        autocomplete="off"
      />
      <input type="password" id="password"
placeholder="Password" />
```

```
<button id="btn-submit"
onclick="prosesAuth()">MASUK</button>

<p id="error-msg" class="error-text"></p>
</div>

<div id="dashboard-section" class="container
hidden">

  <div class="header">
    <h1>Kontrol Objek Virtual</h1>
    <button onclick="keluarAkun()" class="btn-
logout">Logout</button>
  </div>

  <p>Status Sensor Obstacle:</p>
  <div id="visual-lampu" class="lampu mati"></div>
  <h2 id="status-teks">MENUNGGU DATA...</h2>

  <div class="footer">
    <p>Login sebagai: <span id="user-email">-
</span></p>
    <p>Update: <span id="last-update">-</span></p>
  </div>
```

```
    </div>

    <script type="module" src="script.js"></script>
  </body>
</html>
```

Style.css :

```
/* Reset Dasar */
body {
  font-family: "Segoe UI", Tahoma, Geneva, Verdana,
sans-serif;
  background-color: #eef2f3;
  display: flex;
  justify-content: center;
  align-items: center;
  height: 100vh;
  margin: 0;
}

/* Kotak Container Utama */
.container {
  background: white;
  width: 350px;
```

```
padding: 30px;
border-radius: 15px;
box-shadow: 0 10px 25px rgba(0, 0, 0, 0.1);
text-align: center;
transition: 0.3s;
}

/* Menyembunyikan elemen */
.hidden {
  display: none;
}

/* Form Login */
input {
  width: 90%;
  padding: 12px;
  margin: 10px 0;
  border: 1px solid #ddd;
  border-radius: 8px;
  outline: none;
}

button {
  width: 100%;
```

```
padding: 12px;
background-color: #007bff;
color: white;
border: none;
border-radius: 8px;
cursor: pointer;
font-size: 16px;
font-weight: bold;
margin-top: 10px;
}
```

```
button:hover {
    background-color: #0056b3;
}
```

```
/* Pesan Error Login */
```

```
.error-text {
    color: red;
    font-size: 14px;
    margin-top: 10px;
}
```

```
/* Visualisasi Lampu */
```

```
.lampu {
```

```
width: 120px;
height: 120px;
border-radius: 50%;
margin: 30px auto;
transition: all 0.4s ease-in-out;
}

.mati {
background-color: #555;
border: 6px solid #333;
box-shadow: inset 0 0 20px #000;
}

.nyala {
background-color: #56f222;
border: 6px solid #62f055;
box-shadow: 0 0 40px #8cfc6e, inset 0 0 20px
#000000;
}

/* Tombol Logout kecil */
.header {
display: flex;
justify-content: space-between;
```

```
    align-items: center;
    margin-bottom: 20px;
}

.header h1 {
    font-size: 18px;
    margin: 0;
}

.btn-logout {
    width: auto;
    padding: 5px 10px;
    font-size: 12px;
    background-color: #e70303;
    margin-top: 0;
}

.footer {
    font-size: 12px;
    color: #888;
    margin-top: 20px;
}
```

Firmware ESP32 (C++)

Lamp.ino :

```
#include <WiFi.h>

#include <FirebaseESP32.h>

#include "time.h" // Library buat ambil jam internet

// --- KONFIGURASI WIFI ---
#define WIFI_SSID "ID."
#define WIFI_PASSWORD "eko12345"

// --- KONFIGURASI FIREBASE ---
#define FIREBASE_HOST "iot-sensor-saya-default-
rtbd.asia-southeast1.firebaseio.com"
#define FIREBASE_AUTH
"VyZAUfVYX1WuXu21APUHfFzN91V1unn1r2sBmdz6"

// --- SETTING JAM WIB (UTC+7) ---
const char* ntpServer = "pool.ntp.org";
const long  gmtOffset_sec = 25200; // 7 jam x 3600
detik = 25200
const int   daylightOffset_sec = 0;

// --- DEFINISI PIN ---
const int pinSensor = 15;
```



```
// --- OBJEK FIREBASE ---
FirebaseData firebaseData;
FirebaseConfig config;
FirebaseAuth auth;

String statusTerakhir = "";

// --- FUNGSI AMBIL JAM SEKARANG ---
String getJamSekarang() {
    struct tm timeinfo;
    if(!getLocalTime(&timeinfo)){
        Serial.println("Gagal ambil waktu");
        return "Error Waktu";
    }

    // Format jadi: "10/01/2026, 15:45:00"
    char timeStringBuff[50];
    strftime(timeStringBuff, sizeof(timeStringBuff),
"%d/%m/%Y, %H:%M:%S", &timeinfo);

    return String(timeStringBuff);
}
```

```
void setup() {  
    Serial.begin(115200);  
    pinMode(pinSensor, INPUT);  
  
    // 1. Koneksi WiFi  
    WiFi.begin(WIFI_SSID, WIFI_PASSWORD);  
    Serial.print("Connecting to Wi-Fi");  
    while (WiFi.status() != WL_CONNECTED) {  
        Serial.print(".");  
        delay(300);  
    }  
    Serial.println("\nWiFi Connected!");  
  
    // 2. Setting Waktu Internet (WAJIB DITUNGGU)  
    Serial.println("Mengambil Waktu Internet...");  
    configTime(gmtOffset_sec, daylightOffset_sec,  
ntpServer);  
  
    struct tm timeinfo;  
    while(!getLocalTime(&timeinfo)){  
        Serial.print(".");  
        delay(500);  
    }  
    Serial.println("\nWaktu Berhasil Didapat!");  
}
```

```
// 3. Koneksi Firebase
config.database_url = FIREBASE_HOST;
config.signer.tokens.legacy_token = FIREBASE_AUTH;
Firebase.begin(&config, &auth);
Firebase.reconnectWiFi(true);

// --- [TAMBAHAN BARU] KIRIM WAKTU KONEKSI AWAL ---
Serial.println("Mengirim Waktu Start-up ke
Firebase...");

// Ambil jam saat ini
String jamKonek = getJamSekarang();

// Kirim ke path baru, misal:
/Virtual_Object/waktu_mulai_online
Firebase.setString(firebaseData,
"/Virtual_Object/waktu_mulai_online", jamKonek);

Serial.print("Alat Online Sejak: ");
Serial.println(jamKonek);
}

void loop() {
```

```
int deteksi = digitalRead(pinSensor);

if (deteksi == LOW) {
    // --- ADA TANGAN ---
    if (statusTerakhir != "ON") {
        Serial.println("Objek Terdeteksi! -> Kirim
Data");

        // Ambil jam string yang sudah jadi
        String jamWIB = getJamSekarang();

        // Kirim Status
        Firebase.setString(firebaseData,
"/Virtual_Object/status_sekarang", "ON");
        // Kirim Jam yang bisa dibaca manusia
        Firebase.setString(firebaseData,
"/Virtual_Object/waktu_update", jamWIB);

        Serial.print("Update ke Firebase: ON pada ");
        Serial.println(jamWIB);

        statusTerakhir = "ON";
    }
}
```

```
else {  
    // --- KOSONG ---  
    if (statusTerakhir != "OFF") {  
        Serial.println("Kosong -> Kirim Data");  
  
        String jamWIB = getJamSekarang();  
  
        Firebase.setString(firebaseData,  
"/Virtual_Object/status_sekarang", "OFF");  
        Firebase.setString(firebaseData,  
"/Virtual_Object/waktu_update", jamWIB);  
  
        Serial.print("Update ke Firebase: OFF pada ");  
        Serial.println(jamWIB);  
  
        statusTerakhir = "OFF";  
    }  
}  
delay(200);  
}
```